

รายงานโครงงาน

การพัฒนาเกมยานอวกาศสองมิติ Void Gazer ด้วยภาษา Python และ Pygame

จัดทำโดย

1. ปิยะพงศ์ อินทะจินดา

ชั้น CE6921 รหัสนักศึกษา 691540006002-9

2. ณัฐพงศ์ อนาวงศ์

ชั้น CE6921 รหัสนักศึกษา 691540006005-2

เสนอ

อาจารย์ที่ปรึกษา ดร.สรายุทธ กรวิรัตน์

รายวิชา [EN-001-002] การโปรแกรมคอมพิวเตอร์

สถานศึกษา มหาวิทยาลัยกาฬสินธุ์

ภาคเรียนที่ 1 ปีการศึกษา 2569

บทคัดย่อ

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาเกมยานอวกาศสองมิติชื่อ Void Gazer ด้วยภาษา Python และไลบรารี Pygame ให้ผู้เล่นควบคุมการเคลื่อนที่ด้วยแป้น WASD เล็งด้วยเมาส์ ยิงด้วยปุ่มซ้าย และใช้ไนโตรเพื่อเพิ่มความเร็วชั่วคราว เกมประกอบด้วยศัตรู 4 ประเภท ระบบคะแนน พลังชีวิต ไอเทมเสียงประกอบ และการต่อสู้กับบอสที่เป็นเงื่อนไขจบเกม รุ่นที่จัดทำรายงานกำหนดให้บอสปรากฏเมื่อคะแนนถึง 10,000 คะแนน และกำหนดความจุไนโตรเต็มถังเป็นร้อยละ 100

วิธีดำเนินงานใช้การวิจัยและพัฒนาซอฟต์แวร์ เริ่มจากวิเคราะห์ความต้องการ ออกแบบวงจรเกมและสถานะ พัฒนาแยกหน้าที่ตามไฟล์ ทดสอบระบบย่อย และปรับปรุงจากปัญหาที่พบ การตรวจสอบใช้ทั้งการอ่านค่าจากซอร์สโค้ด การทดสอบด้วยช่วงเวลาจำลอง 30 60 และ 120 ครั้งต่อวินาที และการตรวจภาพที่โปรแกรมแสดงผล การทดลองนี้ไม่ได้เก็บแบบสอบถามหรือข้อมูลจากกลุ่มผู้เล่นจริง

ผลการพัฒนาได้เกมที่ทำงานครบวงจร ตั้งแต่เมนูเริ่ม การเล่น การพักเกม การเสียชีวิต ไปจนถึงชัยชนะ บอสมีท่าโจมตีหลักสิบเอ็ดประเภท เลือกท่าตามบริบทของผู้เล่น มีจุดระเบิดสนับสนุนที่เตือนก่อนทำงาน และมีขอบเขตสนามที่ลดขนาดตามระยะการต่อสู้ ผลจำลองพบว่าตัวควบคุมแบบเดิมไม่สามารถผ่านบอสได้ในชุดทดลองสามช่วงเวลา ขณะที่ตัวควบคุมที่รับรู้สนามและจุดระเบิดสามารถชนะได้หนึ่งเงื่อนไข โดยเหลือพลังชีวิตหนึ่งหน่วย ผลดังกล่าวใช้ตรวจสอบความเป็นไปได้ของการเล่นและความท้าทาย ไม่ใช่อัตราชนะของผู้เล่นทั่วไป

ข้อสรุปคือ การใช้เวลาที่ผ่านไประหว่างเฟรม การตรวจชนตามเส้นทาง และการแสดงสัญญาณเตือน ช่วยให้ระบบตรวจสอบและปรับปรุงได้เป็นส่วน ๆ ส่วนความสนุกและความยากที่เหมาะสมยังต้องประเมินกับผู้เล่นจริงในอนาคต ก่อนนำผลไปอ้างอิงในวงกว้าง

คำสำคัญ เกมสองมิติ Python Pygame วงจรเกม ปัญญาประดิษฐ์ในเกม การทดสอบซอฟต์แวร์

สารบัญ

บทคัดย่อ 2
บทที่ 1 บทนำ 4
วัตถุประสงค์และขอบเขตของโครงงาน 5
ประโยชน์ที่คาดว่าจะได้รับและนิยามศัพท์ 6
บทที่ 2 ความรู้พื้นฐาน — แนวคิดเกมและวงจรการทำงาน 7
วงจรเกมและการเปลี่ยนสถานะ 8
พิกัด เวกเตอร์ และการเคลื่อนที่ของผู้เล่น 9
หลักการตรวจชนและการแบ่งเวลาในหนึ่งเฟรม 10
แนวคิดศัตรู ปัญญาประดิษฐ์ และการสื่อสารอันตราย 11
บทที่ 3 วิธีดำเนินการวิจัย 12
เครื่องมือและโครงสร้างโปรแกรม 13
การออกแบบข้อมูลผู้เล่น ศัตรู และคะแนน 14
การออกแบบบอสและสนามต่อสู้ 15
การเพิ่มแรงกดดันและการแก้ปัญหาเชิงระบบ 16
การออกแบบการทดสอบและตัวชี้วัด 17
บทที่ 4 ผลการดำเนินการวิเคราะห์ข้อมูล 18
ผลทดสอบการเคลื่อนที่และไนโตรเต็มถัง 19
ผลการพัฒนาศัตรูและการเพิ่มความยาก 20
ผลการต่อสู้กับบอสและการรับรู้สนาม 21
ผลตรวจสัญญาณเตือน เสียง และเงื่อนไขจบเกม 22
ผลตรวจความถูกต้องของเวลาและภาระการทำงาน 23
บทที่ 5 สรุปและอภิปรายผล 24
อภิปรายผลและความหมายของผลทดลอง 25
ข้อจำกัดและข้อเสนอแนะในการพัฒนาต่อ 26
เอกสารอ้างอิงและแหล่งข้อมูล 27

บทที่ 1
บทนำ

ความเป็นมาและความสำคัญของปัญหา

เกมยานอวกาศเป็นตัวอย่างที่เหมาะสำหรับฝึกพัฒนาโปรแกรมแบบโต้ตอบ เพราะผู้เล่นต้องสั่งการหลายอย่างพร้อมกัน ทั้งการเคลื่อนที่ การเล็ง และการหลบหลีกขณะเดียวกันโปรแกรมต้องปรับตำแหน่งวัตถุ รับเหตุการณ์จากอุปกรณ์ และวาดภาพซ้ำอย่างต่อเนื่อง การสร้างเกมที่ดูเรียบง่ายจึงเชื่อมโยงความรู้เรื่องตัวแปร เงื่อนไข ฟังก์ชัน คลาสเวกเตอร์ และการจัดการทรัพยากรเข้าด้วยกัน [1] [2]

ปัญหาหลักของโครงงานคือทำอย่างไรให้ยานตอบสนองต่อการสั่งงานได้ดีโดยไม่ทำให้ การหมุนยานลดความเร็วโดยไม่ตั้งใจ และทำอย่างไรให้การต่อสู้มีความท้าทายมากกว่าการเคลื่อนที่วน หรือบินหนีในทิศเดิมตลอดเวลา ปัญหาเหล่านี้ต้องแก้ทั้งส่วนควบคุมผู้เล่น รูปแบบการโจมตี การตรวจชนและข้อมูลที่ผู้เล่นเห็นก่อนเกิดอันตราย

Void Gazer จึงออกแบบให้ทิศเคลื่อนที่แยกจากทิศเล็ง ผู้เล่นสามารถถอยหลังหรือเคลื่อนที่ด้านข้างขณะยิงได้ ศัตรูแต่ละประเภทมีหน้าที่ต่างกัน และบอสมีการเปลี่ยนรูปแบบการโจมตี ตามพลังชีวิต การปรับความยากไม่ได้พิจารณาเพียงจำนวนกระสุน แต่พิจารณาว่าผู้เล่นมีข้อมูล และเวลาพอที่จะตัดสินใจเปลี่ยนเส้นทางหรือใช้ทรัพยากรอย่างไนโตรหรือไม่

ประเด็นที่ต้องการศึกษา

โครงงานศึกษาความสัมพันธ์ระหว่างกติกา วงจรการทำงาน และความถูกต้องของระบบโดยใช้ผลการทำงานที่สังเกตหรือวัดจากโปรแกรมได้เป็นหลัก เช่น ระยะเคลื่อนที่ ระยะเวลาเตือน จำนวนวัตถุ และการเปลี่ยนสถานะ เมื่อพบการยิงที่ชนก่อนภาพปรากฏ หรือเหตุการณ์ที่ยังเดินต่อขณะพักเกมจะถือเป็นปัญหาของระบบที่ต้องแก้ไขก่อนประเมินความยากของเกม

วัตถุประสงค์และขอบเขตของโครงงาน

วัตถุประสงค์

1. พัฒนาเกมยานอวกาศสองมิติที่ควบคุมได้ด้วยแป้นพิมพ์และเมาส์พร้อมระบบ คะแนนและเงื่อนไขจบเกมที่ชัดเจน

2. ออกแบบศัตรูและบอสให้มีพฤติกรรมต่างกัน มีสัญญาณเตือน และมีช่องทางหลบหลีกที่ผู้เล่นเรียนรู้ได้

3. ตรวจสอบการเคลื่อนที่ การชน เสียง และการเปลี่ยนสถานะภายใต้ช่วงเวลาจำลองหลายค่าพร้อมปรับปรุง ปัญหาที่ตรวจพบ

ขอบเขตด้านระบบ

เกมเป็นโปรแกรมเล่นคนเดียวบนคอมพิวเตอร์ ใช้ภาพและเสียงที่มีอยู่ในโฟลเดอร์โครงการ หน้าต่างมาตรฐานมีขนาด 1,280 × 800 พิกเซล ตั้งเป้าการทำงานปกติไว้ที่ 60 เฟรมต่อวินาที ผู้เล่นเริ่มด้วยพลังชีวิต 5 หน่วย ศัตรูทั่วไปให้คะแนนต่างกัน และความกดดันของฝูงศัตรูเพิ่มทุก 5,000 คะแนน เมื่อถึง 10,000 คะแนนจะเข้าสู่การต่อสู้กับบอสเพียงตัวเดียว

ระบบที่พัฒนารวมถึงเมนูเริ่ม แถบพลังชีวิต ไนโตร ไอเทมสุ่ม เอฟเฟกต์ตาย เสียงเตือน การพักเกม และหน้าสรุปผล การชนะเกิดจากกำจัดบอสซึ่งมีพลังชีวิต 320 หน่วย และได้รับโบนัส 5,000 คะแนน การตายของผู้เล่นนำไปสู่หน้าสรุปคะแนนและเริ่มเล่นใหม่ได้

ขอบเขตด้านการประเมิน

การประเมินในรายงานมุ่งตรวจสอบความถูกต้องเชิงหน้าที่และพฤติกรรมของโปรแกรม ไม่รวมระบบเครือข่าย การจัดอันดับออนไลน์ การบันทึกคะแนนข้ามการเปิดโปรแกรม หรือการวิเคราะห์ความพึงพอใจทางสถิติ รายงานจะไม่ตีความผลจากตัวควบคุมอัตโนมัติว่าเป็นความสามารถของผู้เล่นจริง

ประโยชน์ที่คาดว่าจะได้รับและนิยามศัพท์

ประโยชน์โดยตรงคือได้ต้นแบบเกมที่สามารถนำไปใช้สาธิตการเขียนโปรแกรมแบบโต้ตอบ และได้โครงสร้างไฟล์ที่อ่านตามหน้าที่ได้ ผู้เรียนสามารถเริ่มจากแก้ค่าความเร็วใน config.py ศึกษาการรับคำสั่งใน player.py แล้วจึงติดตามการเกิดศัตรูและการตรวจชนใน main.py โดยไม่ต้องเริ่มจากเครื่องมือสร้างเกมที่มีองค์ประกอบจำนวนมาก

อีกประโยชน์หนึ่งคือได้ตัวอย่างกระบวนการแก้ปัญหาที่ตรวจสอบได้ เช่น เมื่อต้องการให้เลเซอร์ไม่หยุด เพราะผู้เล่นบินไกล ต้องตรวจเงื่อนไขเปลี่ยนจากการโจมตีไปสู่การไล่ตาม ไม่ใช่เพิ่มความยาวเลเซอร์อย่างเดียว หรือเมื่อเปลี่ยนแดชเป็นไนโตร ต้องปรับทั้งเชื้อเพลิง ภาพไอพ่น เสียงและข้อความบนหน้าจอให้สอดคล้องกัน

คำศัพท์	ความหมายในโครงงาน
เฟรม	ภาพหนึ่งรอบที่โปรแกรมปรับข้อมูลและวาดขึ้นบนหน้าจอ
Delta time หรือ dt	เวลาจริงหรือเวลาจำลองที่ผ่านไประหว่างรอบใช้คำนวณการเคลื่อนที่และ ตัวนับเวลา
ไนโตร	การเพิ่มความเร็วขณะกด Space โดยใช้เชื้อเพลิง และไม่ทำให้ผู้เล่นเป็นอมตะ
สถานะ	ช่วงการทำงาน เช่น เมนู กำลังเล่น พักเกม ชนะ หรือจบเกม
สัญญาณเตือน	ภาพ เสียง หรือข้อความที่แจ้งรูปแบบอันตรายก่อนการโจมตีทำความเสียหาย
สนามบอส	ขอบเขตวงกลมที่ตรึงตำแหน่งเมื่อเริ่มสู้และสร้างความเสียหายหากอยู่นอกเขตนานเกินกำหนด

คำว่า ปัญญาประดิษฐ์ในรายงานนี้หมายถึงกฎและการเลือกพฤติกรรมที่เขียนไว้ในโปรแกรม ไม่ได้หมายถึง แบบจำลองการเรียนรู้ของเครื่องหรือระบบที่ฝึกจากข้อมูลผู้เล่นจำนวนมาก

บทที่ 2
ความรู้พื้นฐาน - แนวคิดเกมและวงจรการทำงาน

ภาษา Python และองค์ประกอบของ Pygame

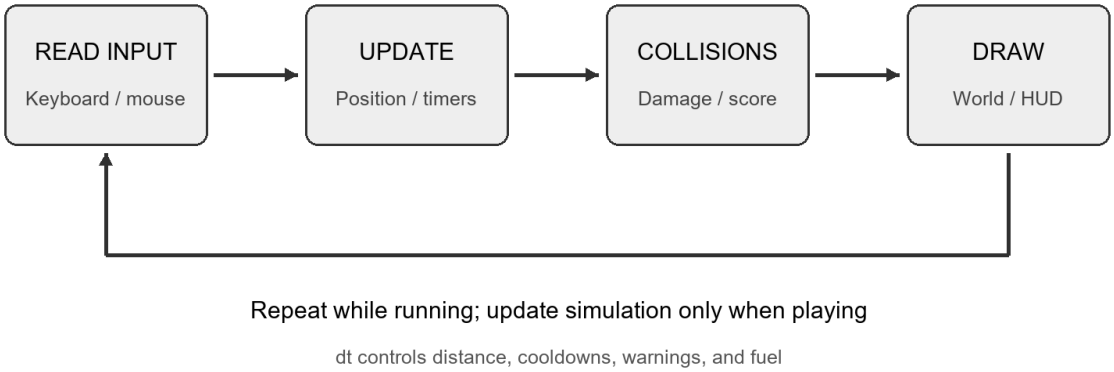
Python มีโครงสร้างข้อมูลและรูปแบบคำสั่งที่เหมาะกับการพัฒนาต้นแบบโครงงาน ใช้รายการเก็บศัตรู และกระสุนใช้พจนานุกรมเก็บข้อมูลการโจมตี และใช้คลาสสำหรับวัตถุที่มีข้อมูลและพฤติกรรมต่อเนื่อง เช่น ผู้เล่นและบอส การแยกไฟล์ช่วยให้แก้ไขพฤติกรรมของศัตรูหนึ่งประเภทได้โดยไม่ต้องเปลี่ยนระบบทั้งหมด [1]

Pygame จัดเตรียมส่วนประกอบสำหรับเปิดหน้าต่าง รับเหตุการณ์ โหลดภาพ วาดรูป และเล่นเสียง ผู้พัฒนายังต้องจัดลำดับวงจรเกมด้วยตนเอง ตัวอย่างวงจรพื้นฐานในเอกสารทางการแสดงการอ่านเหตุการณ์ ปรับภาพ และจำกัดอัตราเฟรม ซึ่งเป็นแนวทางตั้งต้นของโปรแกรมนี้ [2]

ส่วนประกอบ	หน้าที่ที่ใช้จริง
pygame.display และ Surface	สร้างหน้าต่าง เตรียมพื้นที่ภาพ และนำภาพที่วาดเสร็จขึ้นจอ
pygame.event key mouse	อ่านการปิดหน้าต่าง ปุ่มพักเกม การเคลื่อนที่ และตำแหน่งเมาส์
pygame.time.Clock	วัดเวลาระหว่างรอบและจำกัดอัตราเฟรม
pygame.math.Vector2	คำนวณตำแหน่ง ความเร็ว ทิศเล็ง และการหมุนเวกเตอร์
pygame.image และ draw	โหลดสไปรต์ วาดกระสุน เส้นเตือน และแถบแสดงผล
pygame.mixer	โหลดเสียง แยกช่องเสียง และหยุดหรือพักเสียงตามสถานะเกม

โครงงานเลือกใช้เฉพาะส่วนที่จำเป็นโฟลเดอร์หลักจึงประกอบด้วยไฟล์เกมและทรัพยากรภาพเสียง โดยไม่เพิ่มสภาพแวดล้อมเสมือนหรือเครื่องมือสร้างไฟล์จำนวนมากไว้ในโฟลเดอร์ที่ผู้เริ่มต้นต้องเปิดใช้งาน

วงจรเกมและการเปลี่ยนสถานะ



ภาพที่ 2.1 วงจรรับคำสั่ง ปรับข้อมูล ตรวจชน และวาดภาพของเกม

ในหนึ่งรอบ โปรแกรมอ่านเหตุการณ์ก่อน จากนั้นตรวจสถานะปัจจุบัน ถ้าอยู่ระหว่างเล่นและไม่ได้พักเกม จึงปรับผู้เล่น ศัตรู กระสุน และตัวนับเวลา หลังตรวจการชนแล้วจึงอัปเดตคะแนนและพิจารณาว่าต้องเข้าสู่ฉากบอส การตาย หรือชัยชนะ สุดท้ายวาดฉาก วัตถุ และข้อมูลบนหน้าจอแล้วส่งภาพขึ้นจอ [2] [3]

การแยกสถานะช่วยป้องกันเหตุการณ์ที่ไม่ควรเกิดพร้อมกัน ตัวอย่างเช่น เมื่อบอสรับกระสุนนัดสุดท้าย ระบบให้คะแนนและยุติการต่อสู้ทันที ก่อนพิจารณาความเสียหายอื่นในรอบนั้น จากนั้นแสดงแอนิเมชันการตายและเสียงกรีดร้อง จึงเปลี่ยนไปยังหน้าชัยชนะ ผู้เล่นจะไม่ถูกกระสุนที่เหลือทำให้ตายหลังชนะแล้ว

การพักเกมใช้วิธีไม่เพิ่มเวลาจำลองของระบบที่เกี่ยวข้อง ตำแหน่ง วงจรไนโตร สัญญาณเตือน และเอฟเฟกต์จึงหยุดพร้อมกัน เสียงที่กำลังเล่นถูกพักและกลับมาเล่นต่อเมื่อดำเนินเกมต่อ วิธีนี้ต่างจากการหยุดวาดภาพเพียงอย่างเดียว ซึ่งอาจปล่อยให้การโจมตีทำงานอยู่เบื้องหลัง

พิกัด เวกเตอร์ และการเคลื่อนที่ของผู้เล่น

เกมใช้พิกัดโลกเก็บตำแหน่งของวัตถุ ส่วนตำแหน่งบนจอคำนวณเทียบกับผู้เล่นทำให้ยานผู้เล่นอยู่ใกล้กึ่งกลางจอและฉากเลื่อนไปตามการเดินทาง การแยกพิกัดสองระบบนี้ช่วยให้ กำหนดกระสุนและศัตรูในพื้นที่เดียวกัน แม้วัตถุจะยังอยู่นอกจอ

ตำแหน่งใหม่ = ตำแหน่งเดิม + ความเร็ว × dt

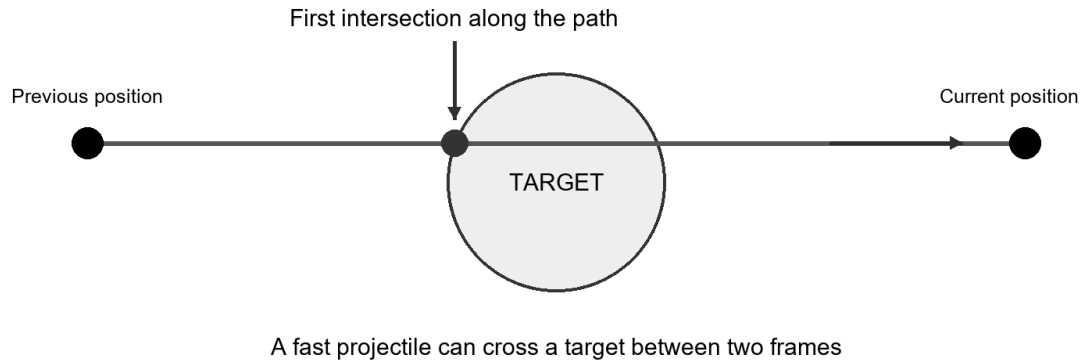
ความเร็วปกติสูงสุดของยานคือ 400 พิกเซลต่อวินาที ทิศที่กด WASD จะถูกปรับให้เป็นเวกเตอร์หน่วยก่อนคูณความเร็วดังนั้นการกดสองทิศพร้อมกันจะไม่ทำให้แนวทแยงเร็วเกินแนวตรง ทิศหมุนของภาพยานคำนวณจากเมาส์แยกต่างหาก ผู้เล่นจึงยิงไปทางหนึ่งขณะถอยหรือเคลื่อนที่ด้านข้างได้ [4]

รายการ	ค่ารุ่นปัจจุบัน	ความหมาย
ความเร็วปกติ	400 พิกเซลต่อวินาที	ความเร็วสูงสุดระหว่างใช้ WASD
ความเร็วไนโตร	820 พิกเซลต่อวินาที	ใช้ขณะกด Space และมีเชื้อเพลิง
ความจุเชื้อเพลิง	100 หน่วย	ถังเต็มแสดงร้อยละ 100
อัตราใช้เชื้อเพลิง	40 หน่วยต่อวินาที	ถังเต็มใช้ต่อเนื่องได้ 2.5 วินาที
การฟื้นเชื้อเพลิง	25 หน่วยต่อวินาที	เริ่มหลังหยุดใช้ 0.75 วินาที

เมื่อเชื้อเพลิงหมด ระบบล็อกการเริ่มเร่งใหม่จนมีอย่างน้อย 20 หน่วยและผู้เล่นปล่อย Space การจำกัดนี้ช่วยแยกการกดค้างจากการเริ่มเร่งครั้งใหม่ ไนโตรให้ความเร็วเพิ่มขึ้น แต่ไม่ยกเลิกความเสียหายจากกระสุน เลเซอร์ หรือการชน

หลักการตรวจชนและการแบ่งเวลาในหนึ่งเฟรม

ถ้าตรวจเพียงตำแหน่งปลายเฟรมกระสุนที่เดินทางเร็วอาจข้ามยานโดยไม่ถูกนับว่าชนโปรแกรมจึงเก็บตำแหน่ง ก่อนและหลังการเคลื่อนที่แล้วตรวจเส้นทางระหว่างสองจุดกับขอบเขตวงกลมของเป้าหมายสำหรับศัตรูที่เคลื่อนที่ด้วย จะเปลี่ยนเป็นการเคลื่อนที่สัมพัทธ์ก่อนตรวจ [4]



ภาพที่ 2.2 การตรวจเส้นทางช่วยตรวจพบการชนที่ตำแหน่งปลายเฟรมเพียงอย่างเดียวอาจมองข้าม

เลเซอร์บอสและเสาพลังงานใช้รูปทรงคล้ายแคปซูล คือเส้นตรงที่มีความหนาและปลายมน การตรวจชนต้องรวมความกว้างของลำแสงกับรัศมีผู้เล่น สำหรับเลเซอร์ที่กวาดหมุน โปรแกรมตรวจการเคลื่อนที่ระหว่างแนวลำแสงเก่าและใหม่ เพื่อไม่ให้ลำแสงข้ามผู้เล่นระหว่างเฟรม

การแบ่งเวลาในเฟรมสำคัญกับการโจมตีที่เริ่มทำงานกลางรอบ เช่น รอยแยกที่เทเลพอร์ตเมื่อผ่านไปร้อยละ 90 ของเฟรมกระสุนที่เพิ่งเกิดจะเริ่มมีผลเฉพาะช่วงเวลาที่เหลือเช่นเดียวกับจุดระเบิดที่ยังไม่หมดเวลาเตือนจะไม่สร้างความเสียหายย้อนหลังหลักการนี้ทำให้ภาพที่แสดงกับผลการชนสอดคล้องกันมากขึ้น

แนวคิดศัตรู ปัญญาประดิษฐ์ และการสื่อสารอันตราย

ศัตรูในเกมมีหน้าที่ต่างกัน สีเหลืองสร้างแรงกดดันด้วยการพุ่งชน สีเขียวเคลื่อนที่โคจรและยิงดักหน้า สีแดงใช้การยิงกระจายต่อเนื่อง ส่วน UFO เป็นเป้าหมายโบนัสที่หลบหนี ไม่โจมตีผู้เล่น การใช้ความแตกต่างเชิงพฤติกรรมช่วยให้ผู้เล่นต้องเลือกเป้าหมายและเส้นทาง แทนการตอบสนองต่อศัตรูที่ทำเหมือนกันทั้งหมด

บอสใช้รายการท่าที่อนุญาตในแต่ละระยะ แล้วสุ่มแบบมีน้ำหนักตามสถานการณ์ เช่น ผู้เล่นอยู่ไกลเคลื่อนที่วนทิศเดิม หรือเข้าใกล้เกินไป การจำท่าที่เพิ่งใช้และการใช้ถุงรายการท่าช่วยลดการโจมตีซ้ำทันทีและทำให้รูปแบบเปลี่ยนไปในแต่ละรอบ วิธีนี้เป็นกฎเชิงโปรแกรม ไม่ใช่การเรียนรู้จากข้อมูลภายนอก

การเพิ่มความท้าทายต้องสื่อสารสิ่งที่กำลังจะเกิดขึ้นด้วย เลเซอร์แสดงพื้นที่ครึ่งวงกลมสีแดงขอบเขตที่ควรหลบออกไป ตัวนับถอยหลัง และเสียงเตือนที่ถี่ขึ้น รอยแยกแสดงตำแหน่งลงจริง ส่วนจุดระเบิดสนับสนุนแสดงวงรัศมีและวงเวลาสะสมก่อนระเบิด ผู้เล่นจึงเห็นเหตุของความเสียหายและมีโอกาสปรับการตัดสินใจ

เสียงและประสิทธิภาพ

ระบบโหลดสไปรต์และเสียงที่ใช้ซ้ำเพียงครั้งเดียว แยกช่องเสียงสำหรับบอส ศัตรู ผู้เล่น สัญญาณเตือน และเครื่องยนต์ไนโตร กระสุนมีอายุการใช้งานและจำนวนสูงสุด ส่วนข้อความที่ไม่เปลี่ยนถูกเก็บไว้ใช้ซ้ำ การออกแบบดังกล่าวลดงานที่ต้องทำในแต่ละรอบ แต่ยังต้องทดสอบบนเครื่องอื่นก่อนอ้างว่ารองรับอัตราเฟรมเดียวกันทุกอุปกรณ์ [5] [6]

บทที่ 3
วิธีดำเนินการวิจัย

รูปแบบการดำเนินงานและการวิเคราะห์ความต้องการ

โครงงานดำเนินงานในลักษณะการพัฒนาและปรับปรุงซอฟต์แวร์เป็นรอบ โดยใช้ความต้องการของผู้ใช้เป็นจุดตั้งต้น แล้วแปลงเป็นพฤติกรรมที่ตรวจสอบได้ เช่น คำขอให้การหมุนไม่ลดความเร็วแปลงเป็นการแยกทิศเล็งออกจากทิศเคลื่อนที่ และตรวจว่าระยะทางในช่วงเวลาเท่ากันยังใกล้เคียงกันเมื่อเปลี่ยนมุมเมาส์

ข้อมูลที่ใช้ประกอบการดำเนินงานมีสามกลุ่ม ได้แก่ ข้อกำหนดการเล่นจากผู้ใช้ ซอร์สโค้ดและทรัพยากรในโครงการ และเอกสารทางการของภาษาและไลบรารี ส่วนข้อมูลผลลัพธ์เกิดจากการรันสถานการณ์ทดสอบ การตรวจภาพหน้าจอ และการบันทึกค่าระหว่างจำลอง ไม่มีการกำหนดประชากรหรือกลุ่มตัวอย่างมนุษย์ เพราะงานระยะนี้ยังไม่ได้ทำการทดลองกับผู้เล่นจริง

ลำดับการดำเนินงาน

1. กำหนดระบบหลัก ได้แก่ ผู้เล่น ศัตรู คะแนน ไอเทม เสียง และเงื่อนไขจบเกม
2. ออกแบบข้อมูลและแยกหน้าที่ในไฟล์ เพื่อให้แก้ไขระบบย่อยได้ชัดเจน
3. พัฒนาระบบพื้นฐานก่อนเพิ่มบอส หลายระยะ และการโจมตีที่ซับซ้อน
4. ทดสอบทั้งค่าปกติและขอบเขต เช่น เชื้อเพลิงหมด กระสุนเกิดกลางเฟรมและพลังชีวิตเหลือหนึ่งหน่วย
5. แก้ไขและตรวจซ้ำ โดยแยกผลการทำงานถูกต้องออกจากผลแพ้ชนะในการต่อสู้

ข้อกำหนดล่าสุดที่ใช้เป็นฐานรายงานคือบอสปรากฏที่ 10,000 คะแนน และไนโตรมีความจุ 100 หน่วย รายงานอธิบายรุ่นนี้โดยตรง จึงไม่ใช้ค่าความจุ 30 หน่วยหรือเป้าหมาย 20,000 คะแนนจากรุ่นก่อนหน้าเป็นข้อสรุปของระบบปัจจุบัน

เครื่องมือและโครงสร้างโปรแกรม

เครื่องมือหลักคือ Python 3.12 และ Pygame 2.6.1 การเปิดเกมบนเครื่องที่ติดตั้งส่วนประกอบแล้วทำได้จาก run_game.bat ส่วน boss_test มีตัวเปิดสำหรับบอสเต็มพลังและบอสหนึ่งพลัง เพื่อแยกการตรวจการต่อสู้ออกจากการตรวจฉากตาย โครงสร้างเน้นไฟล์ชื่อสื่อความหมายและไม่บังคับให้ผู้เริ่มต้นเรียนรู้เครื่องมือเพิ่มเติม ก่อนทดลองเกม

ไฟล์หรือโฟลเดอร์	หน้าที่
main.py	วงจรเกม การเกิดศัตรู การตรวจชน และการเปลี่ยนหน้าจอ
config.py / player.py	ค่าตั้งต้น และการควบคุมยาน ยิง เล็ง ไนโตร พลังชีวิต
yellow_enemy.py / green_enemy.py	ศัตรูไล่ชน และศัตรูเคลื่อนที่โอบล้อม
red_enemy.py / ufo.py	ศัตรูยิงกระจาย และยานโบนัส
boss.py / arena.py	การตัดสินใจของบอส และขอบเขตสนามต่อสู้
projectiles.py / warning.py	กระสุนพิเศษ จุดระเบิด และสัญญาณเตือนอันตราย
effects.py / sounds.py	ภาพเคลื่อนไหว เอฟเฟกต์ และการจัดการเสียง
menu.py / items.py	เมนู สรุปคะแนน และไอเทมสุ่ม
game_sprites / SFX	ไฟล์ภาพและเสียงที่มากับโครงการ

การแยกไฟล์เป็นการแยกความรับผิดชอบ ไม่ใช่การทำให้แต่ละส่วนไม่เกี่ยวข้องกัน ตัวอย่างเช่น boss.py สร้างข้อมูลการโจมตี projectiles.py ปรับการเคลื่อนที่ และ main.py ตรวจผลต่อผู้เล่น จึงต้องรักษาความหมายของข้อมูลเวลาและพิกัดให้ตรงกันทุกส่วน

การออกแบบข้อมูลผู้เล่น ศัตรู และคะแนน

ผู้เล่นเก็บตำแหน่งปัจจุบันและตำแหน่งก่อนหน้า ความเร็ว มุมเล็ง พลังชีวิต เวลาป้องกันหลังถูกโจมตี เชื้อเพลิง และรายการเลเซอร์ ส่วนศัตรูเก็บตำแหน่ง พลังชีวิต คะแนน และสถานการณ์โจมตีของตน การเก็บตำแหน่งก่อนหน้าช่วยให้ตรวจเส้นทางการเคลื่อนที่ในหนึ่งเฟรมได้ แทนการตรวจเฉพาะตำแหน่งปลายทาง

ชนิด	พลังชีวิต	คะแนน	บทบาท
เหลือง ระดับ 1	2	100	ไล่ตามและพุ่งตัดเส้นทาง
เขียว ระดับ 3	3	300	โอบล้อมและยิงคาดการณ์
แดง ระดับ 5	5	600	เคลื่อนหลบแนวเล็งและยิงกระจาย
UFO โบนัส	2	1,000	บินผ่านหลบเลเซอร์และหนีเมื่อหมดเวลา
บอส Void Reaver	320	5,000	เป้าหมายสุดท้ายของเกม

ระบบคะแนนใช้แยกความคืบหน้าของรอบปัจจุบันออกจากสถิติสูงสุดในช่วงเปิดโปรแกรม ความยากคำนวณจากคะแนนปัจจุบันทุก 5,000 คะแนน และรีเซ็ตเมื่อเริ่มรอบใหม่ การเพิ่มระดับทำให้ศัตรูทั่วไปเคลื่อนที่และโจมตีเร็วขึ้น เกิดถี่ขึ้น และเพิ่มจำนวนที่อยู่ในฉาก แต่ไม่ได้เพิ่มพลังชีวิตของศัตรูทุกชนิด

เกมปกติจะเข้าสู่บอสเมื่อถึง 10,000 คะแนน ดังนั้นช่วงศัตรูทั่วไปที่ผู้เล่นพบจริงมีระยะก่อน 5,000 และหลัง 5,000 คะแนน ส่วนสูตรรองรับระดับสูงกว่านั้นไว้ในโค้ด เมื่อเริ่มฉากบอสจะล้างศัตรู กระสุน และไอเทมที่ค้างอยู่ เพื่อเริ่มเงื่อนไขการต่อสู้ที่ชัดเจน

การออกแบบบอสและสนามต่อสู้

บอสแบ่งเป็นสามระยะตามสัดส่วนพลังชีวิต โดยระยะที่สองเริ่มเมื่อเหลือประมาณสองในสาม และระยะที่สามเมื่อเหลือประมาณหนึ่งในสาม ช่วงท้ายที่เหลือไม่เกินร้อยละ 15 จะลดเวลาพักระหว่างการโจมตีอีกครั้ง การเปลี่ยนระยะล้างกระสุนเก่าและมีช่วงเตรียมตัวสั้น ๆ เพื่อไม่ให้การเริ่มรูปแบบใหม่ซ้อนกับอันตรายเดิมโดยไม่แจ้ง

กลุ่มการโจมตี	ท่าที่ใช้	การตัดสินใจที่ต้องการจากผู้เล่น
ไล่กดดัน	Homing Fan / Charge	เปลี่ยนทิศและไม่บินเส้นตรงนานเกินไป
ปิดพื้นที่	Void Ring / Void Mines	เลือกช่องว่างและคำนึงถึงเวลาระเบิด
โจมตีหลายแนว	Spiral / Crossfire	อ่านแนวกระสุนและหลบจุดตัด
โจมตีเป็นช่วง	Starfall / Ion Storm	ออกจากทางอุกกาบาตและช่องพลังงาน
เปลี่ยนตำแหน่ง	Rift Ambush	ออกจากจุดลงที่แสดงไว้ล่วงหน้า
ไปและกลับ	Reaper Blades	เผื่อทางกลับของใบมีดหลังผ่านรอบแรก
กวาดพื้นที่	Deathray	ข้ามขอบพื้นที่เตือนก่อนเลเซอร์กวาด

สนามเริ่มด้วยรัศมี 720 พิกเซล และค่อยลดไปยัง 660 และ 600 ตามระยะบอส ผู้เล่นยังบินออกนอกเขตได้ แต่เมื่ออยู่นอกต่อเนื่อง 0.85 วินาทีจะเริ่มเสียพลังชีวิตตามช่วงป้องกันหลังถูกโจมตี สนามจึงจำกัดการหนีไปไกลโดยไม่ดึงตำแหน่งยานกลับทันที และไม่แก้ความเร็วหรือเชื้อเพลิงของผู้เล่น

การเพิ่มแรงกดดันและการแก้ปัญหาเชิงระบบ

ปัญหาที่พบจากการทดลองก่อนปรับบอสคือการเคลื่อนที่แบบเดิมซ้ำ ๆ สามารถหลบหลายท่าได้ การปรับรุ่นนี้จึงเพิ่มการสังเกตการวิ่งหนีและการหมุนรอบบอส แล้วให้น้ำหนักท่าที่เหมาะกับสถานการณ์ โดยยังคงถุงรายการท่าและไม่เลือกท่าเดิมซ้ำทันที วิธีนี้ทำให้การเปลี่ยนทิศและการบริหารพื้นที่มีความสำคัญมากขึ้น

จุดระเบิดสนับสนุนทำงานแยกจากท่าหลัก บอสทำเครื่องหมายตำแหน่งที่คาดว่าผู้เล่นจะผ่าน และเพิ่มจุดเสริมตามระยะ จุดเหล่านี้เตือน 0.85 วินาทีก่อนทำความเสียหายเป็นเวลา 0.35 วินาที มีรัศมี 72 พิกเซล จำนวนที่สร้างต่อรอบเพิ่มจาก 2 เป็น 3 และ 4 จุด พร้อมช่วงห่างลดจาก 2.7 เป็น 2.1 และ 1.6 วินาที

ก่อนเพิ่มจุดเสริม โปรแกรมตรวจตำแหน่งหลบตัวอย่างรอบพื้นที่เพื่อไม่ให้วงเตือนปิดทางเลือกทั้งหมด ตามแบบตรวจนั้น อย่างไรก็ตาม การตรวจตำแหน่งตัวอย่างไม่ใช่ข้อพิสูจน์ว่าทุกสถานการณ์หลบได้แน่นอน เพราะยังมีการโจมตีหลักและตำแหน่งกระสุนจริงร่วมด้วย จึงต้องใช้ผลทดลองประกอบการปรับค่าต่อไป

ตัวอย่างปัญหาที่แก้ไข

1. รักษาเลเซอร์กวาดให้จบแม้ผู้เล่นบินไกลหรือบอสข้ามเกณฑ์เปลี่ยนระยะ
2. ระบุตำแหน่งรอยแยกก่อนลงจริง และไม่คิดความเสียหายตลอดเส้นทางวาร์ป
3. นับเวลาเกิดและช่วงทำงานของกระสุนภายในเฟรม เพื่อไม่ให้โจมตีก่อนภาพเริ่มทำงาน
4. ยุติความเสียหายและการเกิดวัตถุทันทีเมื่อบอสตาย พร้อมป้องกันการเก็บไอเทมหลังผู้เล่นตาย

การออกแบบการทดสอบและตัวชี้วัด

การทดสอบแบ่งเป็นการตรวจหน้าที่ การตรวจขอบเขตเวลา และการทดลองพฤติกรรมการต่อสู้ การตรวจหน้าที่เปรียบเทียบผลกับกติกาที่กำหนด ส่วนการทดลองต่อสู้บันทึกผลแพ้ชนะ เวลา และพลังชีวิตที่เหลือเพื่อดูผลของกลยุทธ์ ไม่ใช้การแพ้ของตัวควบคุมเป็นหลักฐานว่าโปรแกรมทำงานผิด

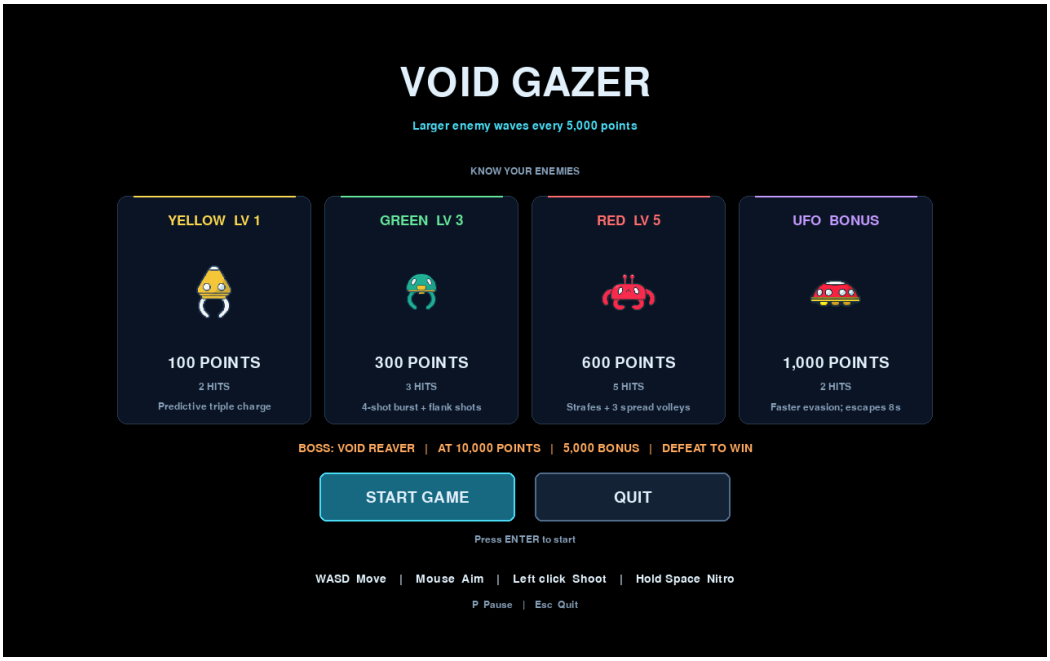
หัวข้อทดสอบ	ตัวแปรหรือข้อมูลที่บันทึก	เกณฑ์พิจารณา
การเคลื่อนที่	ระยะทาง ความเร็ว และมุมเล็ง	ทแยงไม่เร็วเกิน และหมุนเล็งไม่ลดความเร็ว
ไนโตร	เชื้อเพลิง เวลาเร่ง และการปลดล็อก	เต็ม 100 ใช้ 40 ต่อวินาที และไม่เริ่มซ้ำขณะกดค้างหลังหมด
การโจมตี	ตำแหน่งก่อนและหลัง ช่วงเวลาที่มีผล	ไม่ชนก่อนเกิดหรือหลังพ้นช่วงทำงาน
บอสและสนาม	คะแนน ระยะบอส และเวลานอกเขต	เริ่มที่ 10,000 และมีเวลาผ่อนผันก่อนเสียเลือด
การจบรอบ	คะแนน สถานะ และการเล่นเสียง	ให้คะแนนครั้งเดียว แล้วหยุดต่อสู้
ภาระของวัตถุ	จำนวนกระสุนและการลบวัตถุ	อยู่ในเพดาน และลบเมื่อหมดอายุ

การจำลองใช้ช่วงเวลา 1/30 1/60 และ 1/120 วินาที เพื่อหาความคลาดเคลื่อนจากขนาดของเฟรม การเปลี่ยนช่วงเวลาอาจเปลี่ยนผลการต่อสู้ได้จากลำดับการตัดสินใจและการสุ่ม แม้ระบบเวลาพื้นฐานถูกต้อง ดังนั้นต้องตรวจค่าต่อเนื่อง เช่น ระยะทาง แยกจากผลปลายทางอย่างการชนะบอส

ตัวควบคุมอัตโนมัติใช้พลังชีวิตและเชื้อเพลิงตามกติกาปกติในชุดต่อสู้ ส่วนการทดสอบความต่อเนื่อง 90 วินาทีเติมพลังชีวิตระหว่างจำลองเพื่อให้ไปถึงทุกระยะ ชุดหลังใช้หาข้อผิดพลาดและนับวัตถุเท่านั้น ไม่ใช้ยืนยันความยากหรือการผ่านเกม

บทที่ 4
ผลการดำเนินการวิเคราะห์ข้อมูล

ผลการพัฒนาหน้าจอและวงจรการเล่น



ภาพที่ 4.1 เมนูรุ่นปัจจุบัน แสดงคะแนนศัตรูและเป้าหมายบอส 10,000 คะแนน

ผลการพัฒนาได้เมนูเริ่มที่แสดงศัตรูสี่ชนิด คะแนน พลังชีวิต และความสามารถย่อ ผู้เล่นตรวจวิธีควบคุมได้ก่อนเริ่ม โดยใช้ WASD เคลื่อนที่ เมาส์เล็ง ปุ่มซ้ายยิง และ Space ใช้ไนโตร ปุ่ม P ใช้พักเกม ส่วนการเริ่มรอบใหม่อยู่ในหน้าสรุปผล จึงไม่มีปุ่ม R รีเซ็ตระหว่างกำลังเล่น

ระหว่างเล่น หน้าจอแสดงคะแนน พลังชีวิต ความยาก จำนวนศัตรู และปริมาณไนโตร แถบพลังชีวิตเปลี่ยนสีเมื่อเหลือน้อย และมีแถบย่อยใต้ยาน เมื่อตายจะแสดงคะแนนสุดท้าย สถิติสูงสุด ของช่วงเปิดโปรแกรม และเวลาเล่น ผู้เล่นเลือกเล่นใหม่หรือกลับเมนูได้โดยไม่ต้องปิดโปรแกรม

ภาพหน้าจอในบทนี้บันทึกจากโปรแกรมรุ่นที่ใช้จัดทำรายงาน ภาพระหว่างสู้บอส ใช้ตัวเปิดทดสอบบอสเต็มพลัง จึงแสดงข้อความ BOSS TEST บน HUD การใช้ตัวเปิดดังกล่าวมีจุดประสงค์ เพื่อเข้าตรวจฉากที่ต้องการโดยตรง ไม่ได้แสดงว่ารอบทดลองสะสมคะแนนจากศัตรูทั่วไปครบ 10,000 คะแนนแล้ว

ผลทดสอบการเคลื่อนที่และไนโตรเต็มถัง

การตรวจล่าสุดประกอบด้วยข้อยืนยัน 35 ข้อ ครอบคลุมค่ากำหนด การเคลื่อนที่ ไนโตร การพักสนามจุดระเบิด และการจบการต่อสู้ ผลผ่านครบทั้ง 35 ข้อ การตรวจนี้เป็นการตรวจตามสถานการณ์ที่กำหนด ไม่ได้หมายความว่าปราศจากข้อผิดพลาดทุกกรณี ตารางต่อไปเป็นค่าที่วัดจากการเร่งต่อเนื่องตั้งแต่เต็มถังจนหมด โดยไม่ยิงและไม่ใช้ไอเทม

ช่วงเวลาจำลอง	เวลาเร่ง	ระยะที่วัดได้	เชื้อเพลิงปลายทาง
1/30 วินาที	2.5 วินาที	2,050 พิกเซล	0 หน่วย
1/60 วินาที	2.5 วินาที	2,050 พิกเซล	0 หน่วย
1/120 วินาที	2.5 วินาที	2,050 พิกเซล	0 หน่วย

ค่าที่วัดได้สอดคล้องกับความเร็ว 820 พิกเซลต่อวินาทีคูณเวลา 2.5 วินาที และสอดคล้องกับเชื้อเพลิง 100 หน่วยหารด้วยอัตราใช้ 40 หน่วยต่อวินาที ผลจากทั้งสามช่วงเวลาเท่ากันในความละเอียดที่บันทึก แสดงว่าการแบ่งเวลาเมื่อต้องใช้เชื้อเพลิงที่เหลือเพียงบางส่วนของเฟรมทำงานตามที่กำหนด

หลังหมดถัง การกด Space ค้างไม่ทำให้ไนโตรเริ่มซ้ำ แม้เชื้อเพลิงฟื้นเต็ม 100 หน่วยแล้ว เมื่อปล่อยปุ่มและมีเชื้อเพลิงถึงเกณฑ์จึงเริ่มใหม่ได้ การทดสอบยังยืนยันว่าการใช้ dt เท่ากับศูนย์ไม่เปลี่ยนตำแหน่งเชื้อเพลิง หรือเวลารอฟื้น และการหมุนเมาส์ไม่ลดขนาดความเร็วที่คงที่ 400 พิกเซลต่อวินาที

การกดทแยงให้ขนาดความเร็วเท่ากับการกดทิศเดียว ส่วนการชะลอหลังปล่อยปุ่มยังมีระยะสั้นตามค่าความเร่งและแรงหยุด การรายงานระยะไนโตรจึงแยกจากระยะไหลหลังหยุดเร่ง เพื่อไม่รวมพฤติกรรมสองช่วงเป็นตัวเลขเดียว

ผลการพัฒนาศัตรูและการเพิ่มความยาก

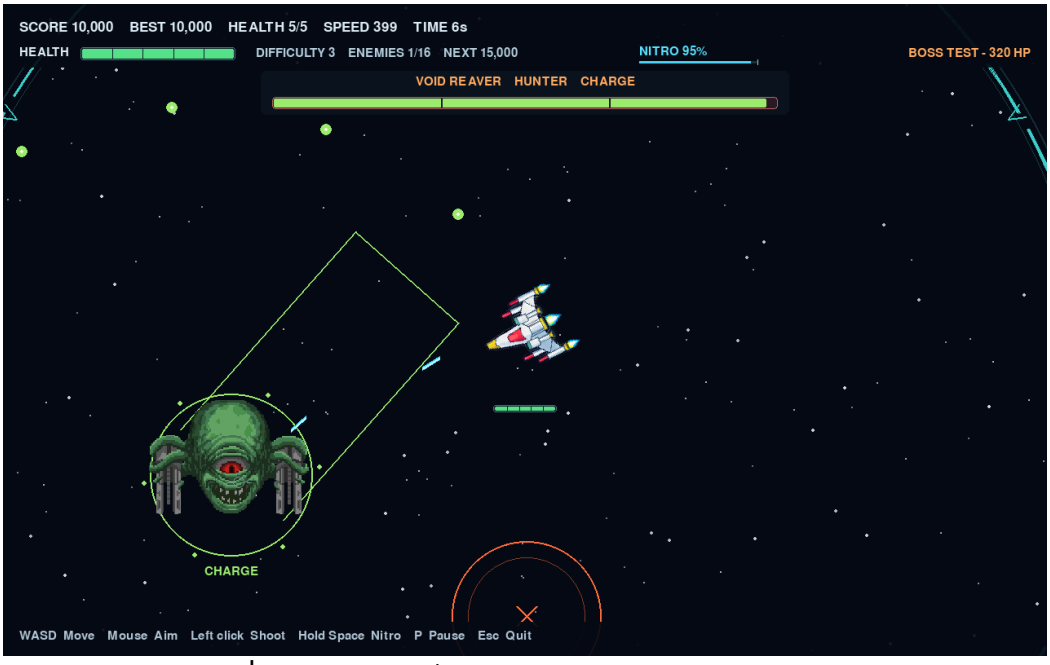
ศัตรูปกติถูกแยกไฟล์และแยกพฤติกรรมตามบทบาท สีเหลืองมีความทนทานต่ำแต่บังคับให้เปลี่ยนเส้นทางด้วยการพุ่ง สีเขียวรักษาระยะและกดดันจากด้านข้าง สีแดงสร้างแนวกระสุนหลายชุด ส่วน UFO เป็นเป้าหมายโบนัสที่ต้องตัดสินใจว่าจะไล่ยิงหรือรักษาตำแหน่งที่ปลอดภัย

ศัตรู	พฤติกรรมที่ทำได้	ข้อจำกัดหรือสัญญาณ
เหลือง	ไล่ที่ 250 พิกเซลต่อวินาที และพุ่งคาดการณ์สามครั้งที่ 850	มีคำเตือนแยกก่อนพุ่งแต่ละครั้ง
เขียว	เคลื่อนโอบและยิงคาดการณ์สี่ครั้ง นัดท้ายเพิ่มกระสุนด้านข้างสองลูก	มีจังหวะเตือนและช่วงพักระหว่างชุด
แดง	เคลื่อนข้างแนวเล็ง และยิงกระจาย 7 5 และ 3 ลูกตามลำดับ	ความเร็วกระสุนเพิ่มแต่จำนวนลดในชุดถัดไป
UFO	บินผ่าน เพิ่มความเร็ว หลบเลเซอร์ และออกจากฉากหลัง 8 วินาที	ไม่โจมตีผู้เล่น และได้คะแนนเมื่อทำลายเท่านั้น

เมื่อเริ่มรอบ ศัตรูอันตรายทำงานด้วยอัตราเริ่มต้นร้อยละ 65 เกิดห่างกันประมาณ 3 วินาที และมีจำนวนสูงสุดเริ่มต้น 8 ตัว เมื่อคะแนนผ่านทุก 5,000 จะเพิ่มเพดานจำนวนและขนาดกลุ่ม พร้อมลดช่วงเกิดตามสูตร การเพิ่มกลุ่มตรวจพื้นที่ว่างก่อนเสมอ จึงไม่เกินเพดานจำนวนที่คำนวณไว้

พลังชีวิตยังคง 2 3 5 และ 2 หน่วยตามชนิด ทำให้คะแนนและจำนวนครั้งที่ต้องยิงอ่านได้จากเมนู การเพิ่มความยากจึงเกิดจากความเร็ว จังหวะ และจำนวนพร้อมกัน เมื่อถึงเป้าหมายบอส ระบบยุติฝูงปกติเพื่อให้ผู้เล่นรับมือชุดโจมตีบอสโดยตรง

ผลการต่อสู้กับบอสและการรับรู้สนาม



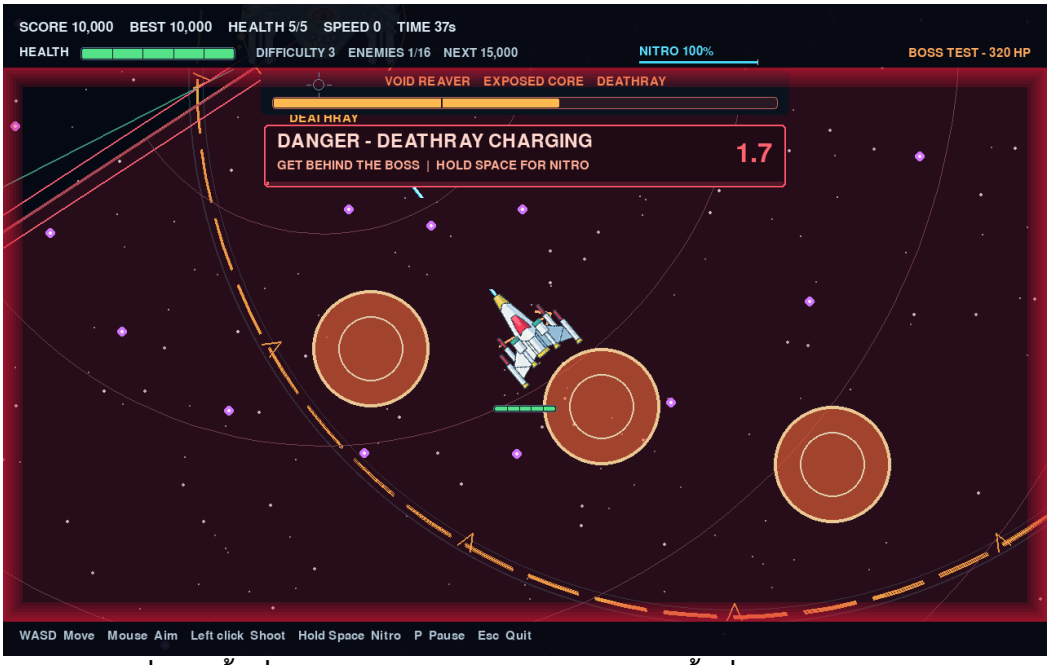
ภาพที่ 4.2 ฉากบอสเต็มพลังและวงเตือนจุดระเบิดสนับสนุน

ตัวควบคุม	30 ครั้ง/วินาที	60 ครั้ง/วินาที	120 ครั้ง/วินาที
แบบเดิม	แพ้ 16.20 วินาที บอสเหลือ 272	แพ้ 17.18 วินาที บอสเหลือ 262	แพ้ 15.91 วินาที บอสเหลือ 269
รับรู้สนามและจุดระเบิด	แพ้ 66.77 วินาที บอสเหลือ 64	แพ้ 50.98 วินาที บอสเหลือ 108	ชนะ 74.33 วินาที ผู้เล่นเหลือ 1

ผลข้างต้นเป็นการจำลองด้วยกติกาปกติ ผู้เล่นเริ่มพลังชีวิต 5 หน่วยและเชื้อเพลิง 100 หน่วย ไม่เติมระหว่างสู้ ผลแสดงว่าการอ่านสนามและวงระเบิดมีผลต่อความอยู่รอด แต่ยังไวต่อช่วงเวลาและเหตุการณ์สุ่ม การชนะหนึ่งเงื่อนไขจึงเป็นหลักฐานว่ากลยุทธ์นั้นผ่านชุดทดลองหนึ่งได้ ไม่ใช่ข้อสรุปว่าผู้เล่นทั่วไปมีอัตราชนะหนึ่งในสาม

การรันผ่านวงจร main.py เพื่อบันทึกภาพอีกครั้งได้ผลแพ้ที่ 69.45 วินาที การทดลองนี้มีลำดับการวาดและเหตุการณ์ต่างจากชุดจำลอง จึงรายงานแยกและไม่รวมเป็นจำนวนซ้ำของชุดข้างต้น

ผลตรวจสัญญาณเตือน เสียง และเงื่อนไขจบเกม



ภาพที่ 4.3 พื้นที่เตือนเลเซอร์กวาด พร้อมเวลา ขอบพื้นที่ และข้อมูลพลังชีวิต

เลเซอร์กวาดแสดงพื้นที่ครึ่งวงกลมก่อนยิง 1.65 วินาที มีเสียงเตือนที่ถี่ขึ้นและขอบสีเขียวเป็นแนวให้ข้ามออกไป จากนั้นกวาด 180 องศาใน 2.3 วินาที ระยะอย่างน้อย 5,000 พิกเซลและขยายเมื่อผู้เล่นอยู่ไกล เงื่อนไขบินหนีไม่ยกเลิกท่าที่กำลังทำงาน และการเปลี่ยนระยะรอให้เลเซอร์จบก่อน

เสียงบอสเรียงตามเหตุการณ์ ได้แก่ roar-329857 ก่อนปรากฏ growl-390285 เมื่อลงถึงฉาก growl-376892 เป็นช่วงระหว่างต่อสู้ และ monster-scream-aggressive เมื่อตาย เสียงศัตรูมาจาก SFX/enemySFX ส่วนเสียงผู้เล่นตายเป็น SFX/plyerSFX/plyerdie.mp3 โดยชื่อโฟลเดอร์คงตามโครงการปัจจุบัน

การตรวจกรณีกระสุนนัดสุดท้ายยืนยันว่าเพิ่มโบนัส 5,000 ครั้งเดียว ลบบอสและกระสุนศัตรู และผู้เล่นไม่รับความเสียหายที่ซ้อนในรอบตัดสินผล จากนั้นจึงเล่นแอนิเมชันตายและแสดงชัยชนะ การตรวจบอสหนึ่งพลังเหมาะสำหรับเหตุการณ์นี้ ส่วนบอสเต็มพลังใช้ตรวจการต่อสู้ครบทุกระยะ

ผลตรวจความถูกต้องของเวลาและภาระการทำงาน

สถานการณ์	ผลที่ตรวจได้
อยู่นอกสนาม 0.84 วินาที	ยังไม่เสียพลังชีวิตจากสนาม
อยู่นอกสนามครบ 0.85 วินาที	เสียพลังชีวิต 1 หน่วย แล้วใช้ช่วงป้องกันปกติ
กลับเข้าสนามหรือพักเกม	กลับเข้าแล้วล้างเวลานอกเขต พักแล้วเวลาไม่เพิ่ม
จุดระเบิดเกิดกลางเฟรม	ช่วงทำงานคิดหลังเวลาเกิดและหลังเวลาเตือนเท่านั้น
ยานผ่านจุดก่อนเริ่มระเบิด	ไม่คิดความเสียหายย้อนหลัง
ยานผ่านระหว่างช่วงระเบิด	คิดความเสียหายตามรัศมีที่กำหนด

การทดสอบต่อเนื่อง 90 วินาทีที่ช่วงเวลา 1/30 1/60 และ 1/120 วินาที เปลี่ยนพลังชีวิตบอสเพื่อเข้าระยะสองและสาม และเติมพลังชีวิตผู้เล่นระหว่างรอบเพื่อให้ตรวจครบ ระบบปรับข้อมูลและวาดภาพได้ตลอดชุดทดสอบ จำนวนกระสุนสูงสุดที่สังเกตในชุดนี้คือ 130 และจุดระเบิดที่เห็นพร้อมกันสูงสุด 4 จุด รัศมีสนามสุดท้ายลดถึง 600 พิกเซล

ในชุดต่อสู้แบบไม่เติมพลัง จำนวนวัตถุโจมตีสูงสุดที่บันทึกได้คือ 132 ต่ำกว่าเพดานบอส 240 แต่ค่านี้เป็นค่าสูงสุดของชุดสถานการณ์ที่รัน ไม่ใช่ค่าสูงสุดที่เป็นไปได้ทั้งหมด ระบบกระสุนทั่วไปยังมีเพดาน 400 และวัตถุที่หมดอายุจะถูกนำออกจากรายการ

การรันทดสอบด้วยไดรเวอร์ภาพจำลองและเวลาจำลองไม่ใช่การวัดอัตราเฟรมจริงของจอหรือประสิทธิภาพทุกเครื่อง รายงานจึงสรุปได้เฉพาะความต่อเนื่องของตรรกะและการควบคุมจำนวนวัตถุ ยังไม่สรุปว่าเกมรักษา 60 เฟรมต่อวินาทีได้บนคอมพิวเตอร์ทุกรุ่น

บทที่ 5

สรุปและอภิปรายผล

สรุปผลตามวัตถุประสงค์

โครงงานพัฒนาเกม Void Gazer ได้ครบวงจรการเล่นตามขอบเขต ตั้งแต่หน้าจอเริ่ม การควบคุมยาน การยิงด้วยเมาส์ ศัตรูหลายชนิด การสะสมคะแนน และการต่อสู้กับบอสจนเกิดผลชนะหรือแพ้ ผู้เล่นเคลื่อนที่ได้ทุกทิศรวมถึงถอยหลัง ทิศเล็งแยกจากการเคลื่อนที่ และมีไนโตรสำหรับเพิ่มความเร็วตามเวลาที่เชื้อเพลิงรองรับ

ด้านศัตรู ได้กำหนดพลังชีวิต คะแนน และความสามารถเฉพาะของศัตรูแต่ละไฟล์ พร้อมเพิ่มความกดดันเมื่อคะแนนผ่านทุก 5,000 คะแนน ส่วนบอสปรากฏที่ 10,000 คะแนน มีพลังชีวิต 320 หน่วย แบ่งสามระยะและมีท่าโจมตีหลักสิบเอ็ดประเภท รวมทั้งจุดระเบิดสนับสนุนและขอบเขตสนามที่ลดลง การชนะบอสเป็นเงื่อนไขจบเกมและให้โบนัส 5,000 คะแนน

วัตถุประสงค์	ผลที่ได้	หลักฐานในรายงาน
พัฒนาเกมที่เล่นครบวงจร	มีเมนู เล่น พัก ตาย และชัยชนะ	ภาพหน้าจอและการตรวจเงื่อนไขจบรอบ
ออกแบบศัตรูและบอส	ศัตรูสี่ชนิดและบอสหลายรูปแบบ	ตารางพฤติกรรมและผลจำลอง
ตรวจและแก้ข้อผิดพลาด	ตรวจเวลา ชน ไนโตร และการตัดสินผล	ข้อยืนยันล่าสุดผ่าน 35 ข้อ

โครงสร้างโปรแกรมยังคงเป็นไฟล์ Python ที่แยกตามหน้าที่และโฟลเดอร์ภาพเสียง ผู้เริ่มต้นเปิดอ่านหรือปรับค่าพื้นฐานได้โดยตรง ขณะที่ส่วนซับซ้อน เช่น การตรวจชนตามเวลา และการตัดสินใจของบอส ถูกจำกัดไว้ในระบบที่เกี่ยวข้อง ไม่กระจายเป็นการแก้ค่าพิเศษหลายจุดโดยไม่มีเหตุผล

อภิปรายผลและความหมายของผลทดลอง

ผลการเคลื่อนที่แสดงว่าการคำนวณด้วยเวลาที่ผ่านไปเหมาะกับระบบทรัพยากรที่ต้องต่อเนื่อง โดยเฉพาะกรณีเชื้อเพลิงไม่พอใช้เต็มเฟรม โปรแกรมแยกเวลาที่เร่งได้จริงออกจากเวลาที่เหลือ ทำให้ระยะเร่งเต็มถังตรงกันในสามช่วงเวลาจำลอง การตรวจทแยงและการหมุนเล็งช่วยยืนยันพฤติกรรมพื้นฐานที่มีผลต่อความรู้สึกควบคุม

ผลต่อสู้ชี้ว่าการปรับความยากด้วยพื้นที่และการโจมตีสนับสนุนเปลี่ยนพฤติกรรมที่ตัวควบคุมต้องรับรู้ แบบเดิมที่เน้นกระสุนและบอสอย่างเดียวแพ้เร็วกว่าแบบที่อ่านสนามและจุดระเบิด การเปรียบเทียบนี้ช่วยอธิบายว่าความกดดันมาจากการตัดสินใจหลายด้านพร้อมกัน แต่ยังไม่เพียงพอจะยืนยันความสนุกหรือความเหมาะสมสำหรับผู้เล่นใหม่

ความยากและความชัดเจนควรพัฒนาควบคู่กัน

การเพิ่มอันตรายโดยไม่เพิ่มข้อมูลอาจทำให้ผู้เล่นไม่เข้าใจสาเหตุที่แพ้ ในงานนี้จึงใช้ภาพเตือนที่สอดคล้องกับบริเวณทำความเสียหาย แสดงเวลานับถอยหลัง และใช้เสียงเฉพาะเหตุการณ์ การแก้รอยแยกให้ตรวจชนเฉพาะตำแหน่งลงจริงเป็นตัวอย่างที่ลดความเสียหายซึ่งอธิบายจากภาพไม่ได้ ขณะที่ยังรักษาความเสี่ยงของท่าโจมตีไว้

ผลแพ้ชนะเปลี่ยนได้เมื่อเปลี่ยนช่วงเวลาหรือเส้นทางจำลอง จึงควรตรวจความถูกต้องด้วยตัวชี้วัดเฉพาะก่อน แล้วค่อยประเมินความยากด้วยการเล่นหลายรอบและหลายกลยุทธ์ หากใช้เพียงการชนะของตัวควบคุมหนึ่งตัว อาจปรับเกมให้ยากเฉพาะกลยุทธ์นั้น และมองข้ามประสบการณ์ของผู้เล่นที่เลือกวิธีต่างออกไป

คำขอให้ไนโตรกลับมาเต็ม 100% เพิ่มทรัพยากรการหลบหลีก แต่สนามบอสทำให้ความเร็วสูงไม่ใช่คำตอบทุกสถานการณ์ ผู้เล่นยังต้องเลือกทิศที่กลับเข้าสนามได้และไม่พุ่งเข้าสู่จุดเตือน จึงเกิดความสัมพันธ์ระหว่างการเคลื่อนที่ เวลา และการเลือกตำแหน่งที่ควรศึกษาต่อกับผู้เล่นจริง

ข้อจำกัดและข้อเสนอแนะในการพัฒนาต่อ

ข้อจำกัดของโครงงาน

รายงานไม่มีผลจากผู้ทดลองจริง จึงยังสรุปความพึงพอใจ ความง่ายในการเรียนรู้ หรือความสมดุลสำหรับกลุ่มอายุใดไม่ได้ ตัวควบคุมอัตโนมัติรับข้อมูลตำแหน่งวัตถุโดยตรงซึ่งต่างจากมนุษย์ที่อ่านภาพบนจอและมีวิธีคาดการณ์จำกัด จึงไม่สามารถแทนการเล่นของมนุษย์ได้ครบทุกด้าน

การทดสอบส่วนใหญ่เน้นความถูกต้องของตรรกะและใช้สถานการณ์ที่กำหนด ยังไม่ได้เปรียบเทียบฮาร์ดแวร์หลายรุ่นหรืออุปกรณ์ควบคุมหลายแบบ ระบบเสียงมีวิธีทำงานต่อเมื่อโหลดเสียงไม่สำเร็จ แต่ยังต้องตรวจความดัง ความชัดเจน และความหน่วงผ่านอุปกรณ์จริงหลายชนิด นอกจากนี้สถิติคะแนนสูงสุดเป็นข้อมูลระหว่างเปิดโปรแกรม ยังไม่มีการบันทึกถาวร

แนวทางพัฒนาต่อ

1. ทดลองกับผู้เล่นจริงโดยกำหนดความยินยอม วิธีเล่น และตัวชี้วัดให้ชัดเจน เช่น เวลาเรียนรู้ จำนวนครั้งที่แพ้ และความเข้าใจสัญญาณเตือน
2. เพิ่มตัวเลือกความยากโดยปรับจังหวะพักและจำนวนการโจมตี พร้อมรักษารูปแบบเตือนให้ตรงกัน
3. เพิ่มการตั้งค่าระดับเสียง การปรับปุ่ม และตัวเลือกภาพเตือนที่แยกได้แม้ไม่อาศัยสีเพียงอย่างเดียว
4. วัดเวลาต่อเฟรมและการใช้หน่วยความจำบนเครื่องหลายระดับ ก่อนกำหนดความต้องการขั้นต่ำ
5. เพิ่มการบันทึกคะแนนและค่าตั้งต้นอย่างง่าย พร้อมตรวจไฟล์ข้อมูลเสียหรือไม่มีสิทธิ์บันทึก

ลำดับที่ควรทำก่อนคือทดสอบความเข้าใจสัญญาณเตือนและการควบคุมกับผู้เล่นจริง แล้วจึงปรับความยากจากหลักฐานที่เก็บได้ การเพิ่มท่าใหม่ควรมาพร้อมวิธีตรวจความถูกต้องและการสื่อสารที่ผู้เล่นอ่านได้ เพื่อรักษาความต่อเนื่องของระบบเมื่อโครงการขยายขึ้น

เอกสารอ้างอิงและแหล่งข้อมูล

แหล่งข้อมูลต่อไปนี้ใช้ประกอบแนวคิดภาษา วงจรเกม เวกเตอร์ เหตุการณ์ และเสียง อ้างหมายเลขในเนื้อหาตามรายการนี้ ตรวจสอบแหล่งข้อมูลเมื่อวันที่ 19 ตุลาคม พ.ศ. 2569 โดยสรุปความด้วยถ้อยคำของผู้จัดทำและเชื่อมโยงกับโค้ดของโครงงาน

[1] ภาษา Python.

Python Software Foundation. Python 3.12 Tutorial. เอกสารประกอบภาษา Python รุ่น 3.12. ใช้ศึกษาโครงสร้างภาษา ฟังก์ชัน คลาส และโครงสร้างข้อมูล. https://docs.python.org/3.12/tutorial/

[2] องค์ประกอบและวงจรพื้นฐานของ Pygame

Pygame Community. Pygame Documentation. ใช้ประกอบโครงสร้างการเปิดหน้าต่าง การวาดภาพ และตัวอย่างวงจรเกมพื้นฐาน. https://www.pygame.org/docs/

[3] เวลาและการควบคุมอัตราเฟรม

Pygame Community. pygame.time. เอกสารฟังก์ชันเวลาและ Clock รวมถึงค่าระยะเวลาที่ได้จาก tick. https://www.pygame.org/docs/ref/time.html

[4] เวกเตอร์สองมิติ

Pygame Community. pygame.math. เอกสาร Vector2 สำหรับการคำนวณทิศ ระยะ การปรับขนาด และการหมุนเวกเตอร์. https://www.pygame.org/docs/ref/math.html

เอกสารอ้างอิงและหลักฐานของโครงงาน

[5] การรับเหตุการณ์

Pygame Community. pygame.event. เอกสารคิวเหตุการณ์ การอ่านเหตุการณ์และการจัดการเหตุการณ์ของหน้าต่างและอุปกรณ์รับคำสั่ง. https://www.pygame.org/docs/ref/event.html

[6] เสียงและช่องเล่นเสียง

Pygame Community. pygame.mixer. เอกสาร Sound และ Channel สำหรับโหลดเสียง เล่น หยุด และพักการเล่นเสียง. https://www.pygame.org/docs/ref/mixer.html

[7] ซอร์สโค้ดและทรัพยากรของ Void Gazer

ปิยะพงศ์ อินทะจินดา และ ณัฐพงศ์ อนาวงศ์. โครงงานเกม Void Gazer. ซอร์สโค้ดรุ่นที่ใช้จัดทำรายงาน ตรวจเมื่อ 19 ตุลาคม พ.ศ. 2569. ค่ากติกาอ้างจาก config.py พฤติกรรมจาก player.py ไฟล์ศัตรู boss.py arena.py projectiles.py และวงจรหลักใน main.py. คำสั่งใช้งานและตัวเปิดทดสอบอยู่ใน README.md และ boss_test.

ภาพและเสียงที่ใช้สร้างเกมมาจากทรัพยากรที่มีอยู่ในโครงการ ได้แก่ game_sprites และ SFX รายงานไม่มีข้อมูลผู้สร้างต้นฉบับของทรัพยากรครบทุกไฟล์ จึงไม่ระบุชื่อเจ้าของผลงานหรือเว็บไซต์ที่ไม่ปรากฏในข้อมูลโครงการ ภาพหน้าจอและแผนภาพประกอบรายงานจัดทำจากโปรแกรมและคำอธิบายระบบในงานนี้

การตรวจซ้ำอย่างย่อ

เปิด run_game.bat เพื่อเล่นตามปกติ หรือเปิด boss_test/full_health.bat เพื่อทดสอบบอส 320 พลัง และ boss_test/one_health.bat เพื่อตรวจฉากตายโดยตรง ค่าตั้งต้นต้องเป็น BOSS_SCORE_TARGET = 10000 และ PLAYER_NITRO_FUEL = 100 ผลเชิงตัวเลขในบทที่ 4 มาจากการทดสอบเฉพาะรุ่นและเงื่อนไขที่อธิบายไว้ ไม่ควรนำไปรวมกับผลของรุ่นที่เปลี่ยนกติกาแล้ว